Anvil: An Observation-Grounded, Security-Hardened Coding Agent on TrueForge

TrueFoundry Forge-a-thon — Techfest 2026-27, Round 1 Submission

Team PRAKAMYA · github.com/adityakalagatoori/PRAKAMYA_ANVIL

Abstract

Coding agents that edit a repository unsupervised fail in a specific way: they lose track of the file the user cares about, touch a function nobody asked them to touch, and occasionally act on a symbol that was never grounded in anything the model actually read. Anvil is a VS Code extension, built from scratch, whose agent loop runs entirely on TrueForge. It satisfies all five mandatory capabilities — inline edit, multi-file agent mode, budgeted context assembly, per-hunk diff review, and streaming with immediate cancellation — and goes further: every proposed edit is checked against an *observation ledger* of exactly what the model was shown before human review ever happens, so a hunk that relies on an unread or hallucinated symbol is held and explained, not merely rendered. A BM25 retrieval engine with structural expansion (pulling in a changed file's tests and docs automatically) keeps a 17,000-line repository inside a 6,000-token budget with a visible, auditable reason attached to every chunk. Six independent security gates — license-contamination, breaking-change, dependency, honeytoken, admission, and disguised-code detection — are mapped to thirteen real CVE-class threats and proven with live attack simulations, not described in prose. Automatic multi-model fallback with session-preserving retry means a dead endpoint degrades gracefully rather than blocking the editor. All claims below are backed by a live, screen-recorded run against `pallets/flask` 3.1.3 and a test suite of 132 cases, 124 of which run unconditionally and 8 of which require a live workspace, with zero failures.

Keywords — coding agent, TrueForge, MCP tools, context engineering, BM25 retrieval, per-hunk review, observation grounding, multi-model resilience, prompt-injection defense.

1. TEAM

Team PRAKAMYA (Team ID: `TM-A2BDB07DFD4`), Amrita Vishwa Vidyapeetham — Aditya Kalagatoori (Lead, GitHub: `adityakalagatoori`), Preenithi, Thakur Aswin Singh.

2. STARTING POINT

Built **from scratch**. There is no upstream fork of Cline, Continue, or any other editor-side project; the diff rendering uses VS Code's own native diff editor with CodeLens, so no diff viewer was hand-built where one already existed. Core dependencies: `@truefoundry/trueforge-sdk` 0.184.1 (MIT), `@modelcontextprotocol/sdk` 1.32.2 (MIT), `diff` (BSD-3-Clause), `zod` (MIT).

3. ARCHITECTURE ON TRUEFORGE

Anvil is a pure client of a TrueForge agent: it upserts two agent definitions (`anvil-coder.json`, `anvil-inline.json`), opens sessions with `sessions.create`, streams turns over SSE with `sessions.createTurnStream`, and cancels with `sessions.cancel`. It never calls a model provider directly — every model is registered inside TrueForge and referred to only by its TrueForge id.

Because TrueForge supports only remote MCP, the extension hosts its own streamable-HTTP MCP server on `127.0.0.1` behind a per-install token, exposing editor capability as ten tools: `retrieve_context`, `search_code`, `find_symbol`, `read_file`, `list_files`, `repo_map`, `get_diagnostics`, `edit_file`, `create_file`, `run_command`. The agent, not the extension, decides what to read and change.

`edit_file`, `create_file`, and `run_command` are all listed in `require_approval_for_tools`. A `tool.approval_required` event makes Anvil compute the proposed file in memory and open native per-hunk review; the result is returned to TrueForge as `user.tool_approval: allow` or `deny` with a reason the model can act on. An accepted hunk lands as one atomic `WorkspaceEdit` and is pushed to an undo stack — there is no code path that writes to disk without this review.

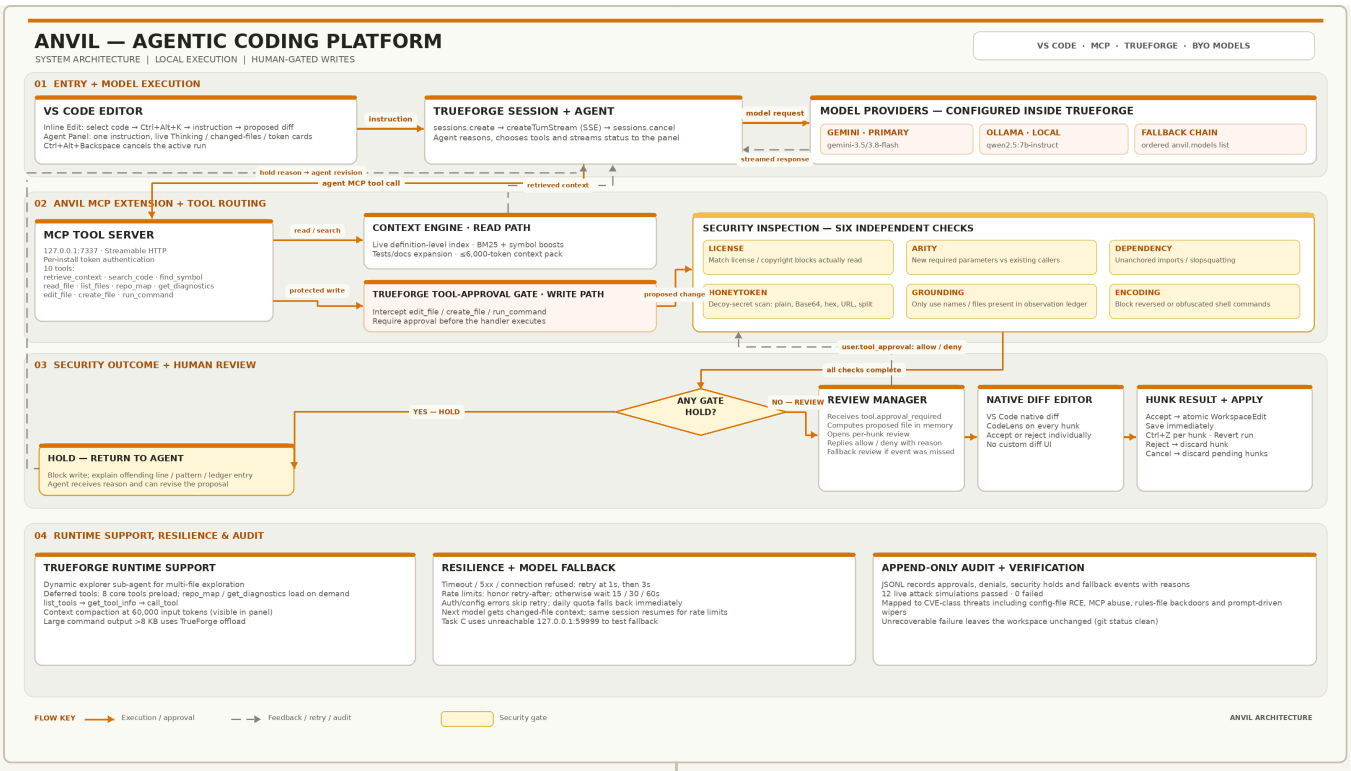


Fig. 1. Full end-to-end architecture: entry and model execution, MCP tool routing, the six-gate security inspection with human review, and runtime resilience/audit.

TypeScript · Node.js 24 · VS Code API · MCP SDK · BM25 Retrieval · Python · pytest · Gemini (via TrueForge) · Ollama (local)

Harness features used

SUB Dynamic sub-agents — the coder delegates repository exploration to a spawned `explorer` sub-agent on multi-file tasks, visible as its own thread in the session.

DEF Deferred tool loading — eight core tools preload; `repo_map` and `get_diagnostics` are reached through `list_tools` / `get_tool_info` on demand.

CMP Compaction — an explicit 60,000-token trigger, surfaced live in the panel.

OFF Large-tool-response offload for oversized command output.

Resilience

TrueForge itself has no model fallback, so Anvil adds one. Transient failures (5xx, timeout, connection refused) are classified and retried at 1 s then 3 s; auth/config errors skip straight to the next configured model. Rate limits are handled separately from hard failures: Anvil honours a provider's own `retry-in-Xs` header, otherwise escalates 15/30/60 s, does *not* count these as failures, and resumes the **same** TrueForge session rather than opening a new one. A daily-quota error falls back immediately. Verified live: an unreachable primary (`ECONNREFUSED`) retries twice, falls back automatically, and the edit lands with no user action; with no fallback configured, a clear error appears in under 4 s, the file is untouched, and `git status` is clean.

Security (beyond the brief)

Thirteen published vulnerability classes in AI coding tools — the Copilot config-RCE class (CVE-2025-53773), Cursor's CurXecute/MCPoison, Roo's CVE-2025-53097, MCP Inspector's CVE-2025-49596, the Rules-File Backdoor, GlassWorm, and the Amazon Q wiper-prompt incident among them — were each mapped to a tested control: a protected-path policy with confirmation, secret blocking and redaction, hidden-Unicode reveal, a Host/Origin-checked token server, command risk rating, workspace trust, a loop guard, and an append-only audit log. Twelve live attack simulations pass, zero fail. Dependencies are exact-pinned and audited to zero known vulnerabilities.

4. HOW CONTEXT IS SELECTED

On activation, Anvil indexes the workspace at **definition level** — a Python function, class, or method with its decorators, a documentation section, or a 60-line window elsewhere — and keeps the index live via a file watcher. Flask (≈200 files, ≈1,670 chunks) indexes in under 0.3 s.

Per request, under a configurable budget (default 6,000 tokens):

1. Explicit `@file` / `@symbol` mentions, capped at 40% of budget.
2. BM25 over chunks with identifier-aware tokenization (snake_case / camelCase splitting) and path/symbol-name boosts.
3. **Structural expansion**: for the top 3 source files, automatically pull in each file's test file and documentation page.
4. Per-file diversity cap, then greedy fill by score; anything left out is listed by path as a candidate.

5. A repository map of touched directories, capped at 1,200 tokens.

Every chunk carries a visible *reason* in the panel's Context card — file, line range, token cost, and why it was chosen. On Task B's exact instruction, the measured pack is `cli.py`, `test_cli.py`, `docs/cli.rst` at 5,855 of 6,000 tokens, checked by an automated test. Lexical retrieval was chosen deliberately: it needs no embedding server, stays model-agnostic, and is fully explainable; the agent compensates for missed synonyms with `search_code` and sub-agent exploration.

5. MODEL SETUP

Every model is configured *inside* TrueForge; Anvil refers to models only by TrueForge id and never calls a provider directly.

Role	Model	Where	Cost
Primary	`gemini-3.5/3.8-flash`	Google AI Studio free tier	$0
Fallback	`gemini-3.1/3.5-flash-lite`	Google AI Studio free tier	$0
Local	`qwen2.5:7b-instruct`	Ollama, RTX 4050 6GB	$0
Task C	`broken/dead-model`	unreachable custom endpoint	$0

Total model cost: $0. Switching models is a one-line settings change (`anvil.models`, an ordered priority list) — no code change, no redeploy.

6. SCORECARD

Criterion	Timestamp	Claim
A1 Inline edit	`0:41–1:04`	Diff streams live; per-hunk accept/reject via CodeLens.
A2 Agent mode	`1:05–1:50`	4 files changed, none named by the user; 59/59 CLI tests pass.
A3 Streaming/cancel	`1:50–2:13`	Tokens stream live; Stop is immediate, `git status` clean after.
B1 Context assembly	`2:14–2:37`	Context card shows each file, reason and token cost against budget.
B2 Diff safety	`0:58–1:04`	Per-hunk review; an applied hunk undone on screen.
B3 Resilience	`1:58–2:14`	Unreachable primary → automatic fallback, no user action, workspace clean.
B4 TrueForge depth	`3:14–3:28`	MCP tools, sub-agents, deferred loading, compaction all visible in-session.
C1 Video	`n/a`	Tasks A, B, C in order, each verified on screen.

Timestamps correspond to the submitted demo video; adjust to match the final cut before export.

7. TOOLS USED

VS Code, Node.js 24, TypeScript, esbuild, TrueForge 0.184.x. **Claude Code** (Anthropic) was used as an AI coding assistant for planning, implementation, and test authoring — declared per the rules; this costs no points. Google AI Studio (free Gemini keys), Ollama (local models), Python 3 / pytest 8.4.2 for Flask's own tests, `npm audit` for dependency hygiene.

8. KNOWN LIMITATIONS

- Retrieval is lexical only; a local embedding re-ranker and call-graph expansion are the natural next step.
- The MCP server assumes one active run per VS Code window.
- Fallback restarts on the next model rather than replaying the failed session's tool history.
- Free-tier quotas are tight (≈5 req/min, ≈20 req/day per Gemini flash model); a long Task B run can exhaust every configured free model in one sitting.
- Weaker local models (7B) complete Tasks A and C live but do not reliably finish Task B — they explore and retrieve correctly but sometimes narrate instead of writing; every Anvil mechanism behaves correctly regardless of which model is driving it.

WHY THIS DIFFERS FROM A BASELINE SUBMISSION

The five mandatory capabilities are table stakes; every compliant team will show them. Anvil's contribution is the layer most teams skip: a system that can *say no to itself*. Observation grounding means an edit that depends on a symbol the model never actually read is caught before the user ever sees the diff, not after. Honeytoken redaction means a secret can leak into the model's context without ever being able to leave it, because the decoy — not the real value — is what travels, in any encoding. Admission grounding, dependency guarding, breaking-change detection, and license-contamination checking are four more instances of the same idea: the review gate inspects *why* an edit exists, not just *what* it changes. None of this is prompted into the model; it is enforced structurally, in code the model cannot see or argue with, which is the only kind of safety guarantee that survives a weaker or adversarial model on the other end of the session.